

FINAL PROJECT REPORT



TutorConnect: A Web-Based Application for Connecting Tutors and Students

**PREPARED BY:
FRAMEWORK PROGRAMMING (D)**

**BRADLEY WIDJAJA
5025231036**

Chapter 1 - Introduction

1.1 Overview

TutorConnect is a web application for managing the connection between students and tutors, and for handling educational materials. The system provides tools for student and tutor registration, profile management, material upload and access, application tracking, and student reviews. It is designed to be a streamlined platform for finding and managing private tutoring relationships and shared learning resources.

TutorConnect is built using NestJS on the backend and Next.js on the frontend, supported by PostgreSQL, Prisma ORM, JWT authentication, and a document processing pipeline involving Sharp, Tesseract, and Google Gemini for structured material/document analysis.

1.2 Purpose

The purpose of this system is to offer a reliable and structured platform for facilitating and managing student-tutor relationships. Instead of relying on informal searches or unverified credentials, TutorConnect centralizes all information in a consistent format. It ensures accurate matching, tracks the status of applications, and provides a central repository for learning materials while maintaining clear records of reviews and activity.

1.3 Key Features

TutorConnect supports the following major features:

- User authentication with access and refresh tokens.
- Dual User Roles: Student and Tutor registration and profile management.
- Material upload (by tutors) and access (by students).
- Application creation (by students to tutors) and status tracking.
- Review submission by students for tutors.
- Profile creation with optional document uploads and advanced searching.
- Platform can send email notifications for events like application status changes, new reviews, etc., using Nodemailer.

- Students and Tutors can filter and search Tutors based on criteria such as name, subject, price.

1.4 Technologies Used

1.4.1 Frontend

- **Next.js 16** : Framework for building the user interface with server-side rendering & routing.
- **React 19** : Component-based UI library.
- **TypeScript 5** : Type-safe JavaScript for scalable frontend development.
- **TailwindCSS 4** : Utility-first CSS framework.
- **@radix-ui/react-slot** : UI composition utility.
- **framer-motion** : Animation library for smooth transitions.
- **lucide-react** : Icon library.
- **clsx, tailwind-merge, class-variance-authority** : Styling helpers.
- **ESLint 9** : Code linting.
- **PostCSS, Autoprefixer** : CSS processing.

1.4.2 Backend

- **NestJS 11** : A scalable and modular Node.js framework for building backend APIs.
- **Node.js** : A JavaScript runtime environment that executes server-side logic.
- **Prisma ORM** : A modern database toolkit used for querying and managing data.
- **bcrypt / bcryptjs** : Libraries used to securely hash passwords for authentication.
- **JWT & Passport** : Tools for implementing secure token-based authentication.
- **Class-validator** : Ensures that incoming request data follows expected rules.
- **Class-transformer** : Transforms and sanitizes data before processing.
- **Multer** : Middleware for handling file uploads such as profile images.
- **Nodemailer** : A library for sending emails like verification and notifications.
- **RxJS** : Provides reactive programming utilities used by NestJS.
- **Reflect-metadata** : Supports decorators used within NestJS architecture.

1.4.3 Testing

- **Jest** : A testing framework used to run unit and integration tests.
- **ts-jest** : Allows Jest to run tests written in TypeScript.
- **Supertest** : Used for testing HTTP endpoints in the backend API.
- **ESLint** : Ensures code consistency and enforces coding standards.
- **Prettier** : Automatically formats code for readability.
- **TypeScript ESLint** : Integrates TypeScript with ESLint.
- **ts-node** : Allows running TypeScript directly without compiling.
- **ts-loader** : Loads TypeScript files in build tools like Webpack.
- **tsconfig-paths** : Enables cleaner import paths in TypeScript.
- **Babel React Compiler** : Helps optimize React components during build.

Chapter 2 - User Stories

2.1 Authentication (AUTH)

- As a User, I want to register securely as either a Student or Tutor, so that I can access the platform with the correct role.
- As a User, I want to log in using valid credentials, so that I can access my account.
- As a User, I want to receive access and refresh tokens, so that I can stay authenticated without repeatedly logging in.

2.2 User Account Management (UPDATE)

- As a User, I want to update my personal information, so that my profile remains accurate and personalized.

2.3 Student Features – Tutor Search & Applications (READ & CREATE)

- As a Student, I want to search and filter Tutors by subject, rating, and price, so that I can find suitable tutors for my needs.
- As a Student, I want to view detailed Tutor profiles (subjects, reviews, materials), so that I can make informed decisions.
- As a Student, I want to submit an Application to a Tutor, so that I can request tutoring sessions.
- As a Student, I want to track the status of my Applications (Pending, Accepted, Rejected), so that I know whether my request is approved.

2.4 Tutor Features – Application & Student Management (READ & UPDATE)

- As a Tutor, I want to view all incoming Applications, so that I can review requests from Students.
- As a Tutor, I want to Accept or Reject Applications, so that I can manage who becomes my Student.

- As a Tutor, I want to view a list of my current Students, so that I can manage my active tutoring relationships.

2.5 Tutor Features – Learning Materials (CREATE, UPDATE, DELETE)

- As a Tutor, I want to upload learning Materials (PDFs, notes, resources), so that my students can access structured learning content.
- As a Tutor, I want to update or delete my Materials, so that I can maintain updated and relevant content.

As a Tutor, I want to set access permissions for my Materials, so that only authorized Students can view or download them.

2.6 Student Features – Access Learning Materials (READ & FILTER)

- As a Student, I want to view and download Materials uploaded by my accepted Tutors, so that I can study efficiently.
- As a Student, I want to filter Materials by subject or by tutor, so that I can quickly find relevant content.

2.7 Student Reviews (CREATE & READ)

- As a Student, I want to submit a Review and rating for a Tutor, so that I can share feedback about my learning experience.
- As a Student, I want to view aggregated ratings for Tutors, so that I can evaluate their performance before applying.

2.8 Tutor Student Management (DELETE)

- As a Tutor, I want to remove a Student from my list if they are no longer learning with me, so that my Student list remains up-to-date.

2.9 Dashboard & Analytics

- As a Tutor, I want to view a dashboard summary (total students, applications, reviews), so that I can quickly understand my performance.
- As a Student, I want to see my application summary on the dashboard, so that I can track my engagement.
- As a Tutor, I want to view analytics (subject breakdown, time-series applications), so that I can understand trends in my tutoring activity.

2.10 Notifications

- As a Tutor, I want to receive a notification when a new Student applies, so that I can respond quickly.
- As a Student, I want to receive a notification when my Application is accepted or rejected, so that I can take the next step immediately.

2.11 File Upload & Document Handling

- As a User, I want to upload files securely, so that my personal documents or learning materials are handled safely.
- As a Tutor, I want the system to reject invalid file types, so that only supported formats are stored.

Chapter 3 – System Architecture

3.1 Overall System Architecture

The TutorConnect system adopts a **modular, service-oriented architecture** divided into three main layers. This design ensures scalability, maintainability, and clear separation of responsibilities.

1. Frontend Layer (Next.js + React)

- Handles all user interface logic, routing, and screen rendering.
- Uses a combination of Client Components (interactive UI, forms, states) and Server Components (server-side rendering, data fetching).
- Communicates with the backend through a secure REST API.
- Implements state management through Zustand and may use TanStack Query for server state.

2. Backend Layer (NestJS)

- Implements core business logic, authorization, and data validation.
- Organized into modules such as: Auth, Users, Tutors, Students, Applications, Materials, Reviews, Uploads.
- Provides REST endpoints secured with JWT authentication.
- Uses Prisma ORM for type-safe database queries and schema management.

3. Database Layer (PostgreSQL with Prisma)

- Stores persistent data including users, tutor profiles, applications, materials, reviews, and uploaded documents.
- Managed using Prisma, enabling schema migrations, modeling, and optimized query generation.
- Uses strict foreign key constraints to ensure data consistency.

These layers interact using HTTP/REST, with requests secured using JWT Authorization headers.

3.2 Authentication Flow

The authentication system uses a secure JWT-based Access + Refresh Token mechanism.

Authentication Flow:

1. User registers or logs in.
2. Backend validates credentials and issues:
 - **Access Token** (short-lived)
 - **Refresh Token** (stored in DB for session tracking and revocation)
3. Every protected API request includes:
 - Authorization: Bearer <access_token
4. When the Access Token expires, the client requests a new one using the Refresh Token.
5. Logging out removes the Refresh Token from the database, fully invalidating the session.

This approach ensures **stateless authentication**, long-lived sessions, and strong security.

3.3 File Upload Flow

TutorConnect includes a secure file-upload pipeline for materials, profile images, and documents.

File Upload Flow:

1. User selects a file and uploads it via the frontend.
2. Multer receives and validates the file (MIME type, size).
3. The Upload Module processes the file and links it to the appropriate entity (e.g., user profile or learning material).
4. File metadata is stored in the database, including filename, file type, original name, and owner ID.
5. The file becomes accessible through a secure, controlled URL.

Security measures include file-type whitelisting, size limits, and access control based on user roles.

3.4 Backend Module Architecture

The backend follows NestJS's **modular architecture**, ensuring a clean separation of responsibilities and easy scalability.

Core Modules:

- **Auth Module:** Login, registration, token handling, refresh tokens, logout.
- **Users Module:** User profiles, role handling, and personal data updates.
- **Tutors Module:** Tutor profiles, subject management, availability, analytics.
- **Students Module:** Student-specific operations and dashboards.
- **Applications Module:** Student-to-tutor application workflow.
- **Materials Module:** Uploading, editing, deleting, and retrieving learning materials.
- **Reviews Module:** Tutor ratings and student feedback.
- **Upload Module:** File storage, validation, ownership management.

This modular structure simplifies maintenance and supports future feature expansion.

3.5 Frontend Architecture

The frontend is built using Next.js App Router, enabling a hybrid rendering approach with both server and client components.

Key Architecture Features:

Server Components

- Handle data fetching and reduce client-side bundle size.
- Improve performance through server-side rendering (SSR).

Client Components

- Used for interactive UI (forms, dashboards, buttons).
- Manage local UI state using Zustand.
- Can integrate TanStack Query for server-state caching.

Routing Structure Example:

/dashboard

/dashboard/applications

/dashboard/materials

/tutors/[id]

/students/[id]

/login

/register

This routing structure keeps the interface organized and improves user navigation.

3.6 Database Architecture

The database uses **Prisma ORM** on top of **PostgreSQL**, enabling type-safe and optimized relational queries.

Design Characteristics:

- Entity-based schema for:
 - Users
 - Tutors
 - Students
 - Applications
 - Materials
 - Reviews
 - Uploads

- Refresh Tokens
- Strong foreign key constraints maintain relational integrity.
- Unique constraints on fields such as email, file ID, and user identifiers.
- Transactional workflows ensure atomic updates—for example, accepting an application updates related tables consistently.

Important Entities:

- **Users:** Base user data, roles, credentials.
- **Tutors:** Tutor-specific information such as subjects, experience, ratings.
- **Students:** Student data and activity.
- **Applications:** Linking students to tutors with statuses.
- **Materials:** Learning content uploaded by tutors.
- **Reviews:** Ratings and feedback provided by students.
- **Uploads:** File metadata and owner tracking.
- **Refresh Tokens:** Session persistence for secure authentication.

Chapter 4 - Backend Design

4.1 Overview

The backend of TutorConnect is built using NestJS 11, following a fully modular architecture. Each feature is isolated within its own module, containing controllers, services, DTOs, validation logic, and Prisma data-access layers. The backend exposes RESTful API endpoints, enforces JWT-based authentication, manages file uploads, handles tutor–student applications, processes learning materials, verifies user roles, and maintains system consistency through strict relational rules. Data persistence is handled using Prisma ORM with PostgreSQL as the primary database.

4.2 Module Structure

Each backend module encapsulates a dedicated domain area of the system.

4.2.1 Auth Module

Responsibilities:

- Register Student or Tutor accounts
- Login and logout
- Issue access and refresh tokens
- Hash and store refresh tokens
- Rotate refresh tokens securely
- Enforce protected-route access via JWT guard

Key components:

- AuthController
- AuthService
- JwtAuthGuard
- JwtStrategy
- DTOs: register.dto.ts, login.dto.ts

This module ensures secure authentication and permission enforcement.

4.2.2 Users Module

Responsibilities:

- Update general user profile
- Update name, avatar, phone number
- Manage role-based visibility
- Provide shared utilities for Student/Tutor-specific modules

Key components:

- UsersController
- UsersService

4.2.3 Tutors Module

Responsibilities:

- Create and update tutor profiles
- Manage subjects and hourly rates
- View student lists
- Provide tutor analytics (active students, materials count, ratings)

Key components:

- TutorsController
- TutorsService

Tutor availability, subjects, and dashboard metrics are computed internally.

4.2.4 Students Module

Responsibilities:

- Manage student profiles
- Fetch accepted tutors
- Provide tutor search filters (subject, price, rating)

Key components:

- StudentsController
- StudentsService

4.2.5 Applications Module**Responsibilities:**

- Allow Students to apply to Tutors
- Prevent duplicate pending applications
- Allow Tutors to accept/reject applications
- Maintain relationships between accepted Students and Tutors
- Track application status changes (Pending, Accepted, Rejected)

Key components:

- ApplicationsController
- ApplicationsService

Business rules (e.g., "cannot accept twice", "cannot apply twice") are enforced inside services.

4.2.6 Materials Module**Responsibilities:**

- Upload learning materials (PDFs, files, notes)
- Update metadata (title, subject, tutor)
- Delete materials
- Restrict visibility only to accepted Students

- Provide filtered views by tutor or subject

Key components:

- MaterialsController
- MaterialsService

Material access rules are checked before returning files.

4.2.7 Reviews Module

Responsibilities:

- Students submit reviews and ratings
- Compute tutor rating analytics (average, count)
- Prevent duplicate reviews between a student–tutor pair

Key components:

- ReviewsController
- ReviewsService

4.2.8 Upload Module

Responsibilities:

- Handle file upload for profile pictures and learning materials
- Validate file type and size
- Store files via Multer
- Return secure URLs for access

Key components:

- UploadController
- UploadService

4.3 Guards, Interceptors, and Pipes

4.3.1 Guards

JwtAuthGuard: Validates access tokens for protected endpoints.

Role Guards

Ensure certain endpoints can only be accessed by:

- Students
- Tutors

Relationship Guards

Service-level checks ensure:

- Students only access materials from accepted tutors
- Tutors only manage applications related to them

4.3.2 Interceptors

Response Interceptor: Standardizes API response structure for consistency.

Snake-Case Interceptor: Converts outgoing responses to snake_case if frontend requires it.

4.3.3 Pipes

ValidationPipe: Automatically validates DTO input with [class-validator](#).

Snake-to-Camel Pipe: Converts request body formatting for backend camelCase compatibility.

4.4 Database Layer

4.4.1 Database Access Pattern

TutorConnect uses Prisma Client for type-safe queries.

Each service interacts with the database through dependency-injected Prisma providers.

4.4.2 Main Entities

- User
- Tutor
- Student
- Application
- Material
- Review
- Upload
- RefreshToken

4.4.3 Transactions

Used when multiple updates must occur atomically, such as:

- Accepting an application (creates student–tutor relationship)
- Removing students from tutor lists
- Deleting materials with associated uploads

4.5 Core Backend Logic

4.5.1 Application Validation

Rules enforced:

- Students cannot apply twice for the same tutor
- Tutors cannot accept an already accepted application
- Only the assigned tutor may modify the application status
- Application status transitions follow a strict flow

4.5.2 Material Access Control

A Student may only access materials if:

1. Their application to that tutor has been accepted
2. The material belongs to that tutor

Permissions are always verified before file content or metadata is returned.

4.5.3 Tutor–Student Relationship Management

When an application is accepted:

- Student is added to tutor’s active student list
- Tutor appears in student’s accepted tutor list
- Dashboard counts are recalculated

When a student is removed:

- Relationship entry is deleted
- Materials and reviews remain unaffected

4.5.4 Review Aggregation Logic

Tutor rating = average of all approved reviews. Backend returns:

- average rating
- total reviews
- most recent reviews
- rating distribution

4.5.5 File Upload Logic

Steps:

1. Multer validates file and saves it
2. UploadService stores metadata in DB
3. Returns secure URL to frontend
4. Optional: Old files are deleted when replacing avatars/materials

4.6 Error Handling

Common error responses include:

- **400** – Validation or malformed input
- **401** – Unauthorized (invalid token)

- **403** – Forbidden (not tutor, not student, not owner)
- **404** – Resource not found
- **409** – Conflict (duplicate applications, duplicate reviews)

All error responses include:

- message
- code
- optional details

Chapter 5 – Frontend Design

5.1 Overview

The frontend of TutorConnect is built using Next.js 15 and React 19, utilizing the App Router for routing and modern React features. The interface focuses on clarity, predictable state flow, reusable UI components, and responsiveness across desktop and mobile. Frontend logic is structured around dedicated layers for authentication state, API communication, role-based rendering, and form handling. This ensures consistency and maintainability throughout the application.

5.2 Application Structure

TutorConnect follows the standard Next.js App Router structure with route groups and segmented layouts.

Key Route Segments

- **(auth)**: Login, Register (public access)
- **(app)**: All authenticated pages
- **/tutors**: Tutor listing & search
- **/tutors/[id]**: Tutor detail page (profile, subjects, materials, reviews)
- **/applications**: Student-side application management
- **/dashboard**: Tutor dashboard (students, analytics, materials)
- **/materials**: Student-accessible learning materials
- **/profile**: User profile settings

Routing is primarily handled through Server Components, with Client Components used for interactive flows, forms, and dynamic UI widgets.

5.3 Client and Server Components

TutorConnect uses a hybrid architecture for performance and scalability.

Server Components

Used for:

- Initial data loading
- Rendering static layouts and shells
- SEO-optimized Tutor list pages
- Tutor profile data fetching
- Student dashboard and material lists

Examples:

- Tutor search results
- Tutor detail layout
- Material list pages

Client Components

Used for:

- Forms (React Hook Form)
- Modals, sheets, and dialogs (Shadcn UI)
- Review submission
- Filters (subjects, ratings, price range)
- Application creation flow
- Tutor analytics charts
- Image upload fields

This hybrid approach reduces bundle size while maintaining interactivity.

5.4 State Management

TutorConnect uses two complementary state systems.

5.4.1 TanStack Query (Server State)

Manages all backend-fetched data:

Used for:

- Caching tutor search results
- Application lists (Student/Tutor)
- Material lists
- Reviews
- Tutor dashboard analytics
- Pagination & filtering

Examples:

- `/tutors?subject=MATH&rating>=4`
- `/applications`
- `/materials/tutor/:id`
- `/dashboard/analytics`

This centralizes data-fetching logic and avoids repetition.

5.4.2 Zustand

Handles client-only UI and session state:

Used for:

- User session/token store
- Modal open/close states
- Selected filters
- Local UI preferences
- Temporary selections (selected tutor, selected material)

Zustand's slice pattern keeps components efficient and predictable.

5.5 Data Fetching and API Layer

All frontend API requests run through a global Axios wrapper.

Key features of the API layer:

- Automatically injects access token
- Intercepts 401 Unauthorized responses
- Triggers refresh-token rotation
- Retries the failed request if refresh succeeds
- Logs the user out if refresh fails
- Centralized error logging

This ensures authentication and data consistency without duplicating logic.

5.6 Form Handling

Forms are built with React Hook Form, supported by Zod where needed. The benefits are:

- Minimal re-renders
- Built-in validation and error handling
- Strong type-safety
- Easy integration with Shadcn components

They are usually used in:

- Student/Tutor registration
- Login
- Tutor profile editing
- Learning material upload
- Application submission
- Review submission

Reusable input components (<Input>, <Textarea>, <Select>) provide consistent UI and validation behavior.

5.7 UI Components & Styling

TutorConnect uses:

- **Tailwind CSS v4** for utility-based styling
- **Shaden UI** for structured components
- **Lucide Icons** for clean iconography
- **Dark/Light theme** support with CSS variables

Common UI Components

- Buttons
- Cards
- Dialogs & Sheets
- Dropdowns
- Tables
- Charts
- Tabs
- File upload components
- Filters (subject, rating, hourly rate)

These ensure visual consistency across the platform.

5.8 Page-Level Feature Design

5.8.1 Tutor Listing Page

Displays all tutors with:

- Subject filters
- Rating filters
- Price range slider
- Keyword search
- Pagination
- Quick profile preview modal

5.8.2 Tutor Detail Page

Contains:

- Tutor bio
- Subjects
- Hourly rate
- Reviews and rating summary
- Material previews (if accepted student)
- Apply button for new students

5.8.3 Applications Page

For Students:

- View application statuses
- Cancel pending applications

For Tutors:

- Accept or reject applications
- View student information

5.8.4 Dashboard (Tutor)

Includes:

- Active students
- Material counts
- Rating overview
- Recent reviews
- Quick upload panel for new materials

5.8.5 Materials Page

Student view:

- List of accessible materials

- Filter by subject or tutor

Tutor view:

- Manage their own uploaded materials
- Edit metadata
- Delete materials

5.8.6 Reviews Section

- Review form for students
- Review list with pagination
- Rating distribution chart

5.8.7 Profile Page

- Update name, avatar, phone
- Change role-specific details (Tutor profile fields)

5.9 Mobile Responsiveness

TutorConnect is fully optimized for mobile.

Features:

- Adaptive navigation menus
- Responsive grid layouts
- Scroll-friendly pages
- Mobile-friendly forms
- Support for camera-based material uploads
- Collapsible sections and tabs

Tailwind responsive utilities simplify breakpoints and adaptive layouts.

5.10 Error Handling and UI Feedback

The frontend uses:

- Toast notifications (success/warning/error)
- Inline form validation messages
- Skeleton loaders for slow fetches
- Empty-state indicators
- Error boundaries and fallback UI

This ensures a predictable, user-friendly experience even during failures.

Chapter 6 – Database Design & Schema Specification

6.1 Overview

The TutorConnect platform uses PostgreSQL as its main relational database, managed through Prisma ORM. The schema is designed to support user management, tutor–student interactions, application workflows, learning materials, and review systems. Prisma ensures type safety, schema consistency, and clear relational mapping across all entities.

6.2 Entity Descriptions

6.2.1 Users Entity

Represents all registered system users, regardless of their role.

Relationships:

- One User → zero or one Tutor profile
- One User → zero or one Student profile
- One User → many Refresh Tokens
- One User → many Uploads
- One User → many Reviews

6.2.2 Tutors Entity

Holds tutor-specific information such as subjects taught, years of experience, and hourly rates.

Relationships:

- One Tutor → belongs to one User
- One Tutor → many Applications
- One Tutor → many Reviews
- One Tutor → many Materials

6.2.3 Students Entity

Contains student-specific information.

Relationships:

- One Student → belongs to one User
- One Student → many Applications
- One Student → many Reviews
- One Student → many Materials (optional)

6.2.4 Applications Entity

Manages application submissions from students to tutors.

Relationships:

- One Application → belongs to a Student
- One Application → belongs to a Tutor

Functionality:

- Students submit requests
- Tutors accept or reject
- Tracks status: *Pending, Accepted, Rejected*

6.2.5 Materials Entity

Stores learning materials uploaded by tutors or students.

Relationships:

- One Material → belongs to a Tutor or Student
- One Material → references one Upload record

Purpose:

- Provides downloadable learning resources
- Centralizes student–tutor learning content

6.2.6 Reviews Entity

Contains ratings and written feedback submitted by students for tutors.

Relationships:

- One Review → belongs to a Student
- One Review → belongs to a Tutor

Purpose:

- Enhances tutor credibility
- Helps students choose suitable tutors

6.2.7 Refresh Tokens Entity

Tracks refresh tokens issued to users for session management.

Purpose:

- Supports secure authentication
- Enables rotating refresh tokens
- Maintains long-lived sessions

6.2.8 Uploads Entity

Centralized storage for file metadata across the platform.

Fields may include:

- File path / public URL
- Size
- MIME type
- Owner User ID

Purpose:

- Standardized file metadata handling
- Supports materials, profile photos, and other uploads

6.3 Integrity and Constraint Policies

6.3.1 Foreign Key Constraints

- Deleting a User **does not delete** their historical applications or reviews
- Deleting a Tutor does not remove associated applications or reviews
- Deleting a Student preserves application records
- Deleting Materials cascades to remove linked Upload metadata
- Deleting Users removes linked refresh tokens

6.3.2 Unique Constraints

- Unique email per User
- One Tutor profile per User
- One Student profile per User
- Student may review a tutor only once
- Unique upload IDs

Chapter 7 – Core Logic and Algorithms

7.1 Overview

This chapter explains the essential business logic and algorithms that power TutorConnect. Unlike basic CRUD operations, these processes ensure correct relationships between Students and Tutors, enforce permissions, validate reviews, manage file uploads, and maintain secure authentication flows. The algorithms are designed to ensure consistency, prevent invalid states, and provide a smooth user experience across all interactions within the system.

7.2 Application Lifecycle Logic

The application workflow defines how students apply to tutors and how tutors respond.

7.2.1 Application Submission Logic

When a Student applies to a Tutor, the system enforces the following rules:

- A Student **cannot apply more than once** to the same Tutor if a pending or accepted application already exists.
- A Student must have a valid user role (student).
- The Tutor must exist and must be active.
- The application is automatically created with status: **Pending**.
- Timestamp fields ensure chronological ordering of decisions.

If any rule is violated, the system returns a **409 Conflict**.

7.2.2 Application Decision Logic (Accept/Reject)

When a Tutor makes a decision:

- Only the assigned Tutor can accept or reject an application.
- Accepted applications create a **Tutor–Student relationship**.
- Rejected applications cannot be updated again.

- Accepted applications cannot be re-accepted.
- Status transitions must follow:
 - **Pending** → **Accepted**
 - **Pending** → **Rejected**

Invalid transitions are blocked with a 400 Bad Request error.

7.3 Material Access Control Algorithm

Access to learning materials is restricted and strictly validated.

7.3.1 Access Validation Rules

A Student can only access a material if **all** conditions are true:

1. The Student's application to that Tutor is **Accepted**.
2. The material belongs to the same Tutor.
3. The material is not marked as deleted.

If any condition fails, a 403 Forbidden is returned.

7.3.2 Unauthorized Access Handling

The system checks permissions **before** returning any metadata or file contents. Unauthorized access results in:

403 — “You do not have access to this material.”

This prevents guessing or enumerating material URLs.

7.4 Review Management Logic

The Review module ensures rating accuracy and prevents abuse.

7.4.1 Review Validation Logic

Before creating a review:

- Only Students with an Accepted application can review the Tutor.
- A Student may only review a Tutor once.
- Review scores must fall within a valid range (e.g., 1 to 5).

Any invalid attempt returns 409 Conflict or 400 Bad Request.

7.4.2 Rating Calculation Algorithm

Tutor ratings are calculated dynamically:

$$\text{average_rating} = \text{sum}(\text{all_review_scores}) / \text{number_of_reviews}$$

The backend also returns:

- total_reviews
- recent_reviews
- rating_distribution
- student-specific review status (whether already reviewed)

This ensures frontend dashboards remain accurate and consistent.

7.5 Tutor–Student Relationship Logic

This logic maintains integrity between accepted applications and active tutor–student connections.

7.5.1 Relationship Creation Rules

A relationship is created only when:

- An application transitions from **Pending** → **Accepted**.
- No previous active relationship exists.

The relationship adds:

- The Tutor to the Student's "Accepted Tutors" list
- The Student to the Tutor's "Active Students" list

This allows filtering, messaging, materials access, and analytics.

7.5.2 Relationship Removal Rules

When a Tutor removes a Student:

- The relationship record is deleted.
- Materials remain intact (no cascading deletes).
- Reviews remain to preserve historical data.
- The Student can reapply in the future.

This ensures data integrity without losing history.

7.6 File Upload & Replacement Algorithm

Handles uploads of profile pictures and learning materials.

7.6.1 File Validation Logic

Multer validates:

- MIME type (PDF, images)
- File size limits
- File category (avatar, material, etc.)

Invalid uploads return **400 Bad Request**.

7.6.2 File Replacement Logic

When a file is replaced (e.g., avatar update):

1. New file is uploaded.
2. Metadata is saved in the Uploads table.
3. If replacing an older file, the old file is optionally removed.

4. New file URL is returned to the frontend.

This ensures clean file management and avoids unused storage.

7.7 Authentication & Token Rotation Logic

7.7.1 Access Token Verification Algorithm

For each protected request:

1. JWT access token is extracted from headers.
2. Signature and expiration are validated.
3. The decoded payload is attached to the request context.
4. If invalid, return:

401 — Invalid or expired token

7.7.2 Refresh Token Rotation Algorithm

When an access token expires:

1. Client sends refresh token.
2. System compares hashed token with stored hash.
3. If valid:
 - A new access+refresh token pair is issued.
 - Old refresh token hash is replaced (rotation).
4. If invalid:
 - User is forced to log out.

This prevents token replay attacks and improves session security.

7.8 Error Handling Logic

7.8.1 Validation Errors (400)

Triggered when:

- DTO validation fails
- Invalid status transitions
- Unsupported file formats

7.8.2 Authorization Errors (401, 403)

401 Unauthorized

- Missing or invalid token

403 Forbidden

- Wrong role (e.g., Student accessing Tutor-only endpoint)
- Accessing materials without permission

7.8.3 Conflict Errors (409)

Triggered when:

- Student submits duplicate application
- Student submits duplicate review
- Tutor attempts to accept twice

All error responses include:

- message
- code
- optional details

Chapter 8 – API Endpoints

8.1 Overview

The TutorConnect backend exposes RESTful endpoints for all modules. All protected routes are secured with JWT authentication. The endpoints cover user management, authentication, tutor–student interactions, learning materials, and reviews.

8.2 Authentication Endpoints (/auth)

- **POST /auth/register**: Register as a Student or Tutor
- **POST /auth/login**: Login and receive access and refresh tokens
- **POST /auth/refresh**: Refresh access token
- **POST /auth/logout**: Logout and invalidate refresh token

8.3 User Endpoints (/users)

- **POST /users**: Create a new user
- **GET /users**: Get list of all users
- **GET /users/:id**: Get specific user profile
- **PATCH /users/:id**: Update user profile (name, avatar, phone)
- **DELETE /users/:id**: Delete a user

8.4 Tutor Endpoints (/tutors)

- **GET /tutors/search**: Search tutors by name, subject, and price range
- **GET /tutors/profile**: Get logged-in tutor's profile
- **PATCH /tutors/:id**: Update tutor profile
- **GET /tutors/:email/students**: Get list of tutor's current students
- **DELETE /tutors/students/:studentId** – Remove a student from tutor's list

8.5 Student Endpoints (/students)

- **GET /students/search-tutors:** Search tutors with filters (subject, name, price range)
- **GET /students/profile:** Get logged-in student's profile
- **PATCH /students/:id:** Update student profile

8.6 Application Endpoints (/applications)

- **POST /applications:** Student applies to a tutor
- **DELETE /applications/:id:** Student cancels an application
- **GET /applications/student:** Get list of student's applications
- **GET /applications/tutor:** Get list of applications sent to a tutor
- **PATCH /applications/:id/status:** Tutor updates application status (accept/reject)

8.7 Material Endpoints (/materials)

- **POST /materials/upload/:tutorProfileId:** Tutor uploads learning material
- **GET /materials/tutor/:tutorProfileId:** Get all materials uploaded by a tutor
- **GET /materials/student/:studentProfileId:** Get materials accessible to a student

8.8 Review Endpoints (/reviews)

- **POST /reviews/:tutorId:** Student submits a review for a tutor
- **PATCH /reviews/:reviewId:** Student updates their review
- **DELETE /reviews/:reviewId:** Student deletes their review
- **GET /reviews/tutor/:tutorId:** List all reviews for a tutor
- **GET /reviews/tutor/:tutorEmail/average** – Get average rating for a tutor

8.9 Upload Endpoints (/upload)

- **POST /upload/profile-picture:** Upload profile picture
- **POST /upload/material:** Upload learning material
- **GET /upload/material/:materialId:** Download material (student access)

Chapter 9 – Test Documentation

My test results cover almost 70% of all lines (62%)

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	62.68	39.4	42.34	61.44	
src	92.85	75	33.33	90.9	
app.controller.ts	87.5	75	50	83.33	10
app.module.ts	100	100	100	100	
app.service.ts	80	100	0	66.66	6
src/applications	67.53	50	50	66.17	
applications.controller.ts	84.21	75	50	82.35	31,37,43
applications.module.ts	100	100	100	100	
applications.service.ts	56	40.62	50	55.55	22,31,43,72-122,149,153,1
src/applications/dto	100	75	100	100	
create-application.dto.ts	100	100	100	100	
update-application.dto.ts	100	75	100	100	6
src/auth	70.17	52.3	57.14	68.62	
auth.controller.ts	83.33	75	40	81.25	19,25,31
auth.module.ts	100	50	100	100	13
auth.service.ts	52.45	40.54	57.14	50.84	46,89,109-192
jwt-auth.guard.ts	100	100	100	100	
jwt.strategy.ts	90	60	100	100	11-27
src/auth/dto	100	75	100	100	
jwt-payload.dto.ts	100	100	100	100	
login.dto.ts	100	100	100	100	
refresh-token.dto.ts	100	100	100	100	
register.dto.ts	100	75	100	100	24
src/mail	100	50	100	100	
mail.module.ts	100	100	100	100	
mail.service.ts	100	50	100	100	12-68
src/materials	74.35	42.1	60	71.87	
materials.controller.ts	77.77	38.46	60	75	25-26,37-43
materials.module.ts	100	100	100	100	
materials.service.ts	57.14	50	60	54.54	13-32
src/prisma	100	100	100	100	
prisma.module.ts	100	100	100	100	
prisma.service.ts	100	100	100	100	
src/reviews	70.76	52.38	66.66	72.72	
reviews.controller.ts	90	75	66.66	88.88	52,58
reviews.module.ts	100	100	100	100	
reviews.service.ts	55.26	43.33	66.66	59.37	22,40,70,93,102-146
src/reviews/dto	100	100	100	100	
create-review.dto.ts	100	100	100	100	
update-review.dto.ts	100	100	100	100	
src/students	63.79	47.72	50	62	
students.controller.ts	77.27	56.25	50	75	36-38,54,59

prisma.module.ts	100	100	100	100	
prisma.service.ts	100	100	100	100	
src/reviews	70.76	52.38	66.66	72.72	
reviews.controller.ts	90	75	66.66	88.88	52,58
reviews.module.ts	100	100	100	100	
reviews.service.ts	55.26	43.33	66.66	59.37	22,40,70,93,102-146
src/reviews/dto	100	100	100	100	
create-review.dto.ts	100	100	100	100	
update-review.dto.ts	100	100	100	100	
src/students	63.79	47.72	50	62	
students.controller.ts	77.27	56.25	50	75	36-38,54,59
students.module.ts	100	100	100	100	
students.service.ts	44.82	42.85	50	44	12-33,56,62,66-69,85-94
src/students/dto	100	100	100	100	
create-student.dto.ts	100	100	100	100	
update-student.dto.ts	100	100	100	100	
src/tutors	26.36	16.66	11.11	22.91	
tutors.controller.ts	51.85	37.5	16.66	48	21-33,48,56,63,69-70
tutors.module.ts	100	100	100	100	
tutors.service.ts	10.52	10.71	8.33	7.57	21-284
src/tutors/dto	91.66	50	0	91.66	
create-tutor.dto.ts	88.88	50	0	88.88	11
update-tutor.dto.ts	100	100	100	100	
src/upload	40.4	16.41	21.42	40	
multer.config.ts	30.43	0	0	23.8	8-19,24-28,32-38
upload.controller.ts	60.86	45.45	25	63.15	41-42,60-61,73-76
upload.module.ts	100	100	100	100	
upload.service.ts	27.65	2.85	33.33	30.55	25-110
src/users	55.76	50	16.66	50	
users.controller.ts	73.68	75	16.66	70.58	14,20,26,32,38
users.module.ts	100	100	100	100	
users.service.ts	30.76	25	16.66	25	13-86
src/users/dto	100	75	100	100	
create-user.dto.ts	100	75	100	100	21
update-user.dto.ts	100	100	100	100	

Test Suites: 2 passed, 2 total
Tests: 11 passed, 11 total
Snapshots: 0 total
Time: 22.756 s
Ran all test suites.

Youtube Video:

https://youtu.be/IaZsejRaxkk?si=ng4VOV_pQE6JqwiO